

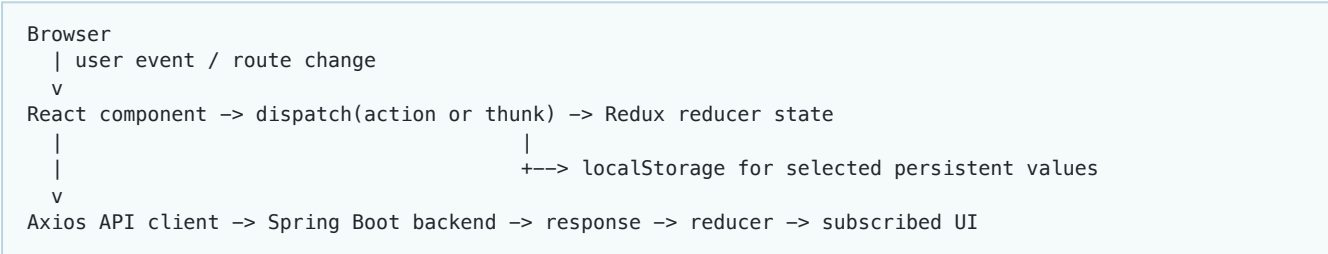
sb-ecom-frontend

Technical architecture, component behavior, and process-flow guide

- Application:** React + Vite ecommerce frontend
- State:** Redux Toolkit with localStorage persistence
- Backend style:** REST API with credentialed Axios requests
- Generated:** September 10, 2026

1. System Overview

This project is a client-side ecommerce application. React components render pages and reusable controls, React Router changes views without full-page reloads, Redux Toolkit coordinates shared state, and async action creators call the backend. Local storage restores selected session data after a browser refresh.



Layer	Files	Technical responsibility
Bootstrap and routing	src/main.jsx , src/App.jsx	Creates the React root, attaches Redux Provider, configures router, global navbar, and toast notifications.
Presentation	src/components/**	Pages, forms, dialogs, cards, loaders, navigation, and checkout screens.
State and commands	src/store/actions/** , src/store/reducers/**	Async API workflows, synchronous state transitions, loading/error flags, and persistence.
Integration	src/api/api.js	Axios base URL and credential configuration.
Helpers/assets	src/hooks/** , src/utils/** , src/assets/**	Query translation, formatting, banner metadata, text truncation, and images.

2. Setup And Runtime

```
npm install
npm run dev      # Vite development server
npm run build    # production build
npm run lint     # ESLint validation
npm run preview  # preview production build
```

Required local environment values:

```
VITE_BACK_END_URL=http://localhost:8080
VITE_STRIPE_PUBLISHABLE_KEY=your_stripe_publishable_key
VITE_FRONT_END_URL=http://localhost:5173
```

`src/api/api.js` creates an Axios instance with `${VITE_BACK_END_URL}/api` as the base URL and `withCredentials: true`. The environment file is local-only and ignored by Git.

3. Startup And Route Processing

`src/main.jsx`

Calls `createRoot`, renders `App`, and wraps it in Redux's `Provider`. Every descendant can therefore call `useSelector` and `useDispatch`.

`src/App.jsx`

Creates the `BrowserRouter`, places the global `Navbar` and toast container around the route content, and maps URLs to pages.

URL	Guard	Page/functionality
/	Public	Home product preview and hero banners.
/products	Public	Search, category filtering, sorting, product cards, and pagination.
/about	Public	Static product/about presentation.
/contact	Public	Contact details and form UI.
/cart	Public	Local shopping cart.
/checkout	Private	Address, payment, summary, and payment steps.
/order-confirm	Private	Post-payment order confirmation.
/login, /register	Public-auth guard	Only unauthenticated users should access the forms.

`src/components/PrivateRoute.jsx`

Reads `state.auth.user`. It returns `Outlet` when the condition is satisfied, otherwise redirects to `/login`. For login/register, the `publicPage` option redirects an existing user to `/`.

```
URL request
-> React Router matches route
-> PrivateRoute reads auth.user
    -> authenticated: render nested page
    -> unauthenticated: Navigate to /login
-> page component mounts and starts its data workflow
```

4. Redux Architecture

Store construction: `src/store/reducers/store.js`

`configureStore` combines `products`, `errors`, `carts`, `auth`, and `payment`. Before store creation, it reads `auth`, `cartItems`, and `CHECKOUT_ADDRESS` from `localStorage` and injects them into `preloadedState`.

Slice	State shape / purpose	Consumers
products	products , categories , pagination	Home, Products, Filter, ProductCard, pagination.
errors	isLoading , errorMessage , category and button flags	Loaders, error pages, disabled buttons, toasts.
carts	cart , totalPrice , cartId	Navbar badge, Cart, ItemContent, checkout/payment.
auth	user , addresses, selected address, clientSecret	PrivateRoute, UserMenu, checkout, payment.
payment	paymentMethod	PaymentMethod, OrderSummary, payment screens.

Action-to-render lifecycle

```

Component event
-> dispatch(thunk or action)
-> thunk sets loading state
-> thunk calls Axios endpoint
-> success: dispatch success action, persist if required
    failure: dispatch error action and show toast/UI error
-> reducer returns new slice state
-> useSelector subscribers render updated UI

```

Reducers

`ProductReducer.js` stores catalog response data and pagination metadata. `errorReducer.js` centralizes request status. `cartReducer.js` handles add, increase, decrease, remove, server-cart hydration, totals, and clear. `authReducer.js` handles user session, addresses, selected address, Stripe secret, logout, and checkout cleanup. `paymentMethodReducer.js` stores the chosen payment method.

Actions: `src/store/actions/index.js`

This file contains product/category fetch thunks; cart add/update/remove operations; sign-in, registration, and logout; address CRUD and selection; server-cart creation/loading; Stripe client-secret creation; and payment confirmation.

5. Home Page Process

`home/Home.jsx`

On mount it dispatches product retrieval. While the request is active it renders loading UI; on failure it renders an error; on success it passes the first eight products to product cards alongside `HeroBanner`.

`home/HeroBanner.jsx`

Builds an autoplay Swiper carousel from banner data exported by `src/utils/index.js`. A slide click navigates to `/products`.

```

Home mounts
-> dispatch fetchProducts(query)
-> GET /api/public/products
-> ProductReducer stores products/pagination
-> Home selects products
-> first eight ProductCard components render
-> HeroBanner independently maps static banner metadata to Swiper slides

```

6. Product Search, Filters, And Pagination

`products/Products.jsx`

Loads categories, invokes `useProductFilter`, selects products and errors from Redux, and renders filter controls, cards, loader/error states, and pagination.

`products/Filter.jsx`

Writes category, keyword, and sort values into URL search parameters. Keyword updates are debounced for 700 ms; clear resets the relevant parameters.

`hooks/useProductFilter.js`

Reads URL search parameters, constructs the backend query, converts the UI's one-based page to backend zero-based `pageNumber`, and dispatches `fetchProducts`. The URL is the source of truth for a reloadable/shareable product view.

`shared/Paginations.jsx`

Changes the `page` query parameter, which triggers the filter hook and a new backend request.

```
User changes filter or page
-> Filter/Paginations updates URLSearchParams
-> useProductFilter observes URL values
-> query string is generated
-> fetchProducts dispatches GET /api/public/products
-> ProductReducer replaces products and pagination
-> Products re-renders cards and page controls
```

7. Product Display And Modal

`shared/ProductCard.jsx`

Displays image, name, price, discount, stock status, and actions. Add-to-cart dispatches the cart thunk with the selected product. A detail action opens the modal. The `about` prop suppresses normal purchase UI on the About page.

`shared/ProductViewModal.jsx`

Headless UI dialog for a product's image, pricing, description, and stock state. It is controlled by the selected product and open/close state from the parent.

```
ProductCard action
-> add to cart: dispatch addToCart(product, quantity, toast)
  -> stock check and duplicate merge
  -> cart reducer updates cart/totalPrice
  -> cartItems written to localStorage
-> details: set selected product + open modal
  -> ProductViewModal renders details
  -> close resets modal state
```

8. Cart Functionality

`cart/Cart.jsx`

Selects `state.carts.cart`, calculates subtotal using `specialPrice * quantity`, renders item rows, and links to checkout or continued shopping.

`cart/ItemContent.jsx`

Renders item details and dispatches increase, decrease, and remove actions. Local quantity display follows the selected item while Redux remains the shared source of truth.

`cart/SetQuantity.jsx`

Reusable control that disables decrement at quantity one and invokes callbacks for both directions.

`cart/CartEmpty.jsx`

Empty-state view with a link back to the store.

```
Add product
-> find matching product in cart
-> if match: increase quantity; else append item
-> validate available stock
-> calculate totalPrice
-> persist cartItems
-> Navbar badge and Cart update

Quantity/remove
-> dispatch cart action
-> reducer changes item list and totalPrice
-> persist new cartItems
-> ItemContent and summary re-render
```

9. Authentication Process

`auth/Login.jsx`

React Hook Form validates username/password, then dispatches `authenticateSignInUser`. The thunk posts credentials, updates loading/error state, stores the returned user, shows a toast, and navigates home.

`auth/Register.jsx`

Validates username, email, and a six-character minimum password, dispatches `registerNewUser`, then navigates to login after successful signup.

`UserMenu.jsx`

Shows account actions for the current user and dispatches logout. Its profile/order links require corresponding application routes to become functional.

```
Login form submit
-> validate fields
-> POST /api/auth/signin
-> auth reducer stores user
-> localStorage.auth stores session response
-> PrivateRoute now allows checkout/order-confirm
-> Navbar changes from Login to UserMenu
```

10. Checkout And Address Management

`checkout/Checkout.jsx`

Controls a four-step Material UI stepper: address, payment method, order summary, and payment. It loads addresses on mount and blocks Next until each required selection exists.

`AddAddressForm.jsx`

Creates or edits addresses through React Hook Form. Edit mode receives an existing address and pre-fills fields; submit dispatches the address save/update thunk.

AddressInfo.jsx

Coordinates address empty state, list, add/edit dialog, and delete dialog.

AddressInfoModal.jsx

Headless UI dialog wrapper that hosts `AddAddressForm`.

AddressList.jsx

Renders selectable addresses and edit/delete controls. Selection dispatches to Redux and writes `CHECKOUT_ADDRESS` to localStorage.

DeleteModal.jsx

Confirms deletion and displays progress while the delete request runs.

```
Checkout mounts
-> GET /api/addresses
-> auth reducer stores address list
-> user selects address
-> selectUserCheckoutAddress updates Redux + CHECKOUT_ADDRESS
-> Next validates selected address
-> payment step becomes available
```

11. Payment And Order Confirmation

PaymentMethod.jsx

Stores Stripe or PayPal selection. If a local cart has no `cartId`, it dispatches server-cart creation before payment continues.

OrderSummary.jsx

Reads selected address, payment method, and cart state to show billing information, line items, quantities, prices, and subtotal.

StripePayment.jsx

Requests the Stripe client secret and creates the Stripe `Elements` context for `PaymentForm`.

PaymentForm.jsx

Renders Stripe's Payment Element, submits payment details, confirms payment with the client secret, and routes to `/order-confirm`.

PaypalPayment.jsx

Displays that PayPal is unavailable; it does not call a payment provider.

PaymentConfirmation.jsx

Reads Stripe redirect query parameters, dispatches order confirmation, clears cart/checkout state after success, and displays the result.

```
Payment sequence
-> choose method
-> if cartId missing: POST /api/cart/create
-> Stripe selected: POST /api/order/stripe-client-secret
-> auth reducer stores clientSecret
-> Elements renders PaymentForm
-> elements.submit() + stripe.confirmPayment(...)
-> Stripe redirects to /order-confirm?payment_intent=...
-> POST /api/order/users/payments/online
-> clear cart and checkout state
-> render confirmation
```

12. API Contract Map

Method	Endpoint	Triggered by	Redux result
GET	/public/products	Home, Products, filter changes	products and pagination
GET	/public/categories	Products mount	categories
POST	/auth/signin	Login	auth.user
POST	/auth/signup	Register	toast/navigation
GET	/addresses	Checkout mount	auth.address
POST/PUT	/addresses , /addresses/{id}	AddAddressForm	address list refresh
DELETE	/addresses/{id}	DeleteModal	address removed
POST	/cart/create	PaymentMethod	carts.cartId
GET	/carts/users/cart	Cart/checkout synchronization	server cart hydration
POST	/order/stripe-client-secret	StripePayment	auth.clientSecret
POST	/order/users/payments/online	PaymentConfirmation	order completion/cleanup

13. Shared UI And Support Files

File	Functionality
shared/Navbar.jsx	Sticky responsive navigation, cart badge, login link, and signed-in user menu.
shared/InputField.jsx	React Hook Form input wrapper with required, min-length, email, and URL validation.
shared/Loader.jsx	Centered RotatingLines loader with optional text.
shared/Spinners.jsx	Small inline SVG spinner.
shared/CustomSkeleton.jsx	Animated placeholder while content loads.
shared/ErrorMessage.jsx	Reusable error message view.
shared/Status.jsx	Configurable status badge with text, icon, background, and text color.
Backdrop.jsx	Dismissible full-screen overlay for open menus/dialogs.

14. Utilities, Assets, And Styles

File/folder	Functionality
utils/constant.js	Imports/exports slider images and banner image collection.
utils/index.js	Static banner metadata consumed by HeroBanner.
utils/formatPrice.js	USD currency formatting, quantity-price calculation, and abbreviated revenue output.
utils/truncateText.js	Truncates text to a configurable maximum, default 30 characters.
src/index.css	Global CSS and Tailwind entry.
src/App.css	Application-level visual styles.
src/assets/	Hero, slider, React, and Vite assets.

15. Complete User Journey

1. Vite starts the app; `main.jsx` mounts Redux Provider and `App`.
2. Store initialization restores session, cart, and selected address from `localStorage`.
3. Router selects Home, Products, or an authentication page based on the URL.
4. Home or Products dispatches catalog requests; reducers populate product state.
5. `ProductCard` adds items; cart reducer checks stock, merges quantities, calculates totals, and persists the cart.
6. Login stores the user; `PrivateRoute` exposes protected checkout routes.
7. Checkout loads addresses, persists the selected address, and validates each step.
8. `PaymentMethod` creates a server cart when necessary; `StripePayment` obtains a client secret.
9. `PaymentForm` confirms the Stripe payment; confirmation sends the result to the backend.
10. Successful order completion clears cart and checkout state and displays the confirmation page.

16. Project File Map

src/	
App.jsx	Router, global Navbar, toaster, routes
main.jsx	React root and Redux Provider
api/api.js	Axios backend client
components/	
About.jsx	About page
Backdrop.jsx	Menu/dialog overlay
Contact.jsx	Contact page/form UI
PrivateRoute.jsx	Auth route guard
UserMenu.jsx	Account menu/logout
auth/	Login and registration
cart/	Cart, empty state, item and quantity controls
checkout/	Address, summary, payment, confirmation
home/	Home and hero carousel
products/	Catalog, filters, pagination
shared/	Reusable UI controls
hooks/useProductFilter.js	URL query to product request hook
store/actions/index.js	Async thunks and action creators
store/reducers/	Redux slices and store setup
utils/	Formatting, banner and text helpers
assets/	Product and banner images

This guide describes the current frontend source tree and its implemented client-side workflows. Backend validation, database behavior, authentication implementation, and payment-provider processing remain in the backend/provider systems.